

Manual de instalación — StudIA

Versión: 1.0 · **Fecha:** 18/09/2026

Proyecto: StudIA — Asistente de estudio sobre corpus académico (PPS · FI-UNLZ)

Este manual cubre las dos formas de poner StudIA en funcionamiento: **instalación desde el instalador** (para usar el sistema) y **compilación desde el código fuente** (para desarrollarlo o corregirlo).

1. Requisitos

1.1 Sistema

	Mínimo	Recomendado
Sistema operativo	Windows 10 64 bits	Windows 11 64 bits
RAM	8 GB	16 GB
Disco (aplicación + índice + modelo)	3 GB	5 GB
Disco (corpus completo, opcional)	—	8 GB
GPU	no requerida	NVIDIA con 6 GB+ de VRAM

Sin GPU el sistema funciona, pero el modelo de chat responde más lento.

1.2 Dependencias externas

StudIA **funciona sin ninguna de estas**: el chat, la búsqueda, las citas y los modos andan igual. Lo que se apaga son funciones puntuales:

Dependencia	Qué habilita	Si falta
Python 3 + librerías	Sumar bibliografía propia; herramientas de ingesta	El botón  no procesa documentos
matplotlib	Gráficos de funciones	El gráfico se muestra como texto
Node.js + mermaid-cli	Diagramas	El diagrama se muestra como texto
Tesseract (con español)	OCR de PDF escaneados	Esos documentos no se pueden indexar
Runtime de Visual C++	Ejecución de <code>llama-server.exe</code>	El servidor del modelo muere al arrancar

La aplicación **detecta cuáles faltan**, indica qué función se pierde en cada caso y ofrece instalarlas con un clic. También se pueden instalar a mano (§3).

2. Instalación desde el instalador

2.1 Instalar la aplicación

1. Ejecutar `StudIA-1.0-Setup.exe` (≈ 1,21 GB).
2. Aceptar la carpeta de destino propuesta o elegir otra.
3. Finalizar.

No requiere permisos de administrador: instala para el usuario actual. La ruta elegida queda registrada en `HKA\Software\StudIA\InstallDir`.

El instalador incluye:

Contenido	Ubicación tras instalar
Aplicación (LlamaCode + StudIA)	carpeta de instalación
Índice del corpus (<code>studia.db</code>)	<code><instalación>\StudIA\</code>
Modelo de embeddings (bge-m3)	<code><instalación>\StudIA\modelos\</code>
Scripts de dependencias y de corpus	<code><instalación>\StudIA\</code>

Por qué los documentos originales no vienen adentro: son 7,7 GB y sólo hacen falta para *abrir* el PDF desde una cita. El texto citado está dentro del índice, así que el chat, las citas, la abstención y los modos funcionan sin ellos.

2.2 Instalar las dependencias

Desde la propia aplicación, cuando avisa que faltan herramientas, o a mano:

```
<instalación>\StudIA\instalar_dependencias.bat
```

Instala Python con sus librerías, Node.js con mermaid-cli, Tesseract con el paquete de español y el runtime de Visual C++. Es **idempotente**: correrlo dos veces no reinstala nada.

Para ver qué falta sin instalar:

```
.\instalar_dependencias.ps1 -SoloRevisar
```

2.3 Copiar los documentos originales (opcional)

Sólo hace falta si se quiere que las citas **abran el PDF**.

```
<instalación>\StudIA\copiar_documentos.bat
```

o con destino propio:

```
.\copiar_documentos.ps1 -Destino D:\Docs
```

Copia el corpus (7,7 GB) desde el medio de entrega a `C:\StudIA_Docs` por defecto.

Por qué un destino de ruta corta. Las carpetas del material académico tienen rutas de más de 260 caracteres, el límite que Windows admite. La copia se hace con `robocopy`, que sí las maneja, pero el destino tiene que ser corto: dentro de la carpeta de instalación se perdían 76 de 2.937 archivos; en `C:\StudIA_Docs`, 2. Por el mismo límite el corpus **no puede** distribuirse dentro del instalador —un `.exe` tampoco puede superar los 4,2 GB—.

2.4 Primera ejecución

1. Abrir **StudIA** desde el menú de inicio.
2. Ir a la sección  **StudIA**.
3. Elegir una materia y preguntar.

El servidor de embeddings se levanta solo al abrir la aplicación y se cierra al salir. La primera respuesta demora más porque el modelo se está cargando.

2.5 Desinstalación

Panel de control → *Aplicaciones* → **StudIA** → Desinstalar. Los documentos copiados a `C:\StudIA_Docs` no se eliminan: hay que borrarlos a mano si ya no se usan.

3. Instalación manual de dependencias

Si se prefiere no usar el script:

```
# Python 3 (o desde python.org marcando "Add to PATH")
winget install --id Python.Python.3.12 --source winget

# Librerías: extracción de texto, gráficos y OCR
pip install pypdf python-docx python-pptx openpyxl numpy matplotlib pypdfium2 pytesseract

# Node.js y mermaid-cli para los diagramas
winget install --id OpenJS.NodeJS.LTS --source winget
npm install -g @mermaid-js/mermaid-cli

# Tesseract con español
winget install --id UB-Mannheim.TesseractOCR --source winget

# Runtime de Visual C++ (lo necesita llama-server.exe)
winget install --id Microsoft.VCRedist.2015+.x64 --source winget
```

Después de instalar algo que modifica el `PATH`, **cerrar y volver a abrir la consola**: el `PATH` de un proceso ya iniciado no se actualiza solo.

4. Compilación desde el código fuente

4.1 Requisitos de desarrollo

Herramienta	Versión
Visual Studio 2022	con <i>Desktop development with C++</i>
Qt	6.x (Quick / QML)
CMake	3.21 o superior
Git	cualquiera
Python 3	para las herramientas y sus pruebas

4.2 Obtener el código

```
git clone https://github.com/MarcosDePalma/UNLZ_Llamacode_StudIA.git
cd UNLZ_Llamacode_StudIA
git checkout feature/studia
```

4.3 Compilar y probar

```
build.bat Release      # compila la aplicación
tests.bat Release     # compila y corre la suite completa
```

La condición para dar por buena una modificación es **build más suite en verde**. Si la máquina no tiene Python, las suites en Python se omiten y el resto igual corre.

4.4 Preparar el índice

La aplicación necesita un `studia.db`. Para generarlo a partir de un corpus propio, ver el **Manual del corpus e ingesta**:

```
python tools/studia/ingest.py --corpus "D:\FACULTAD\DATA" --db "D:\StudIA\studia.db"
python tools/studia/vectorizar.py --db "D:\StudIA\studia.db" --url http://127.0.0.1:8081
```

4.5 Generar el instalador

Requiere [Inno Setup](#) instalado.

```
installer\compilar.bat
```

Genera `dist\StudIA-1.0-Setup.exe`. El script copia además el runtime de MSVC junto al ejecutable —sin eso la aplicación no abre en una PC sin Visual Studio— y actualiza los scripts empaquetados con la versión del repositorio.

5. Verificación de la instalación

Comprobación	Resultado esperado
La aplicación abre y muestra la sección 🎓 StudIA	✓
El selector de materias lista las asignaturas	el índice se encontró
Una pregunta sobre un tema del material devuelve respuesta con citas	recuperación operativa
Una pregunta ajena al material devuelve la respuesta de abstención	control de abstención operativo
El aviso de herramientas no reporta faltantes	dependencias completas
Un click en una cita abre el documento	el corpus está copiado

6. Problemas frecuentes

Síntoma	Causa	Solución
La aplicación no abre, error de DLL	Falta el runtime de MSVC	Instalarlo (§3)
LlamaCode abre pero el servidor del modelo muere sin mensaje	Falta el runtime de Visual C++ para <code>llama-server.exe</code>	Ídem: el ejecutable vive en otra carpeta y no ve las copias locales
El selector de materias aparece vacío	No se encuentra <code>studia.db</code>	Verificar <code><instalación>\StudIA\studia.db</code>
StudIA se abstiene siempre	El índice está vacío o es de otro corpus	Re-indexar y recalibrar el umbral
Las citas no abren el documento	El corpus no fue copiado o está en otra ruta	Ejecutar <code>copiar_documentos.bat</code>
El botón  no procesa	Falta Python	<code>instalar_dependencias.bat</code>
Faltan archivos tras copiar el corpus	Ruta de destino demasiado larga	Usar un destino corto como <code>C:\StudIA_Docs</code>

7. Validación en una PC limpia

El instalador se verificó en la máquina de desarrollo y con un **usuario nuevo de Windows**, que reproduce el estado de perfiles y configuración de una instalación limpia sin instalar nada:

Configuración → Cuentas → Otros usuarios → Agregar cuenta
→ "No tengo los datos de esta persona" → "Agregar un usuario sin cuenta Microsoft"

Eso detecta rutas que apuntan a la carpeta personal del desarrollador, configuración que se creía por defecto y perfiles duplicados. **No** detecta lo que se instala a nivel de máquina (Python, Node, Tesseract, runtime de Visual C++), que ya está presente en el equipo de desarrollo. Para eso hace falta una máquina virtual: el procedimiento completo está en `installer/PROBAR_EN_PC_LIMPIA.md` del repositorio de código.